Enterprise Employee Operations Hub

System Architecture, REST API Specification, & Operations Manual

Workforce Management • Internal Service Desk • Cross-Border Operations

Malayo Tech Operations Team

Version 1.0.0 – October 2026

Executive Overview

The **Employee Operations Hub** is a production-oriented, full-stack enterprise operations platform engineered to manage workforce administration, client delivery engagements, internal IT/HR service desk ticketing, hardware custody tracking, and dual-location cross-border calendar synchronizations (Philippines ↔ Denmark). It features strict Role-Based Access Control (RBAC), multi-tier operational segregation, and dual-database support (zero-config SQLite for developer agility alongside containerized PostgreSQL 16 for production workloads).

Contents

1 Architectural Overview & Technology Stack **3**
- 1.1 High-Level System Topology . . . 3
- 1.2 Technology Stack Breakdown . . . 3
- 1.3 Dual Database Provider Strategy . . . 3

2 Codebase Directory Structure **3**

3 Role-Based Access Control (RBAC) Architecture **5**
- 3.1 Dual-Dimension Conceptual Model . . . 5
- 3.2 Seed Testing Personas Matrix . . . 6
- 3.3 Granular Authorization Flags . . . 6

4 Domain Entities & Data Models **6**
- 4.1 Core Entity Definitions . . . 6
  - 4.1.1 Employee Entity (`Employee.cs`) . . . 7
  - 4.1.2 Department Entity (`Department.cs`) . . . 7
  - 4.1.3 Ticket Entity (`Ticket.cs`) . . . 7
  - 4.1.4 Hardware Custody Item (`HardwareCustodyItem.cs`) . . . 7
  - 4.1.5 Cross-Border Holiday (`Holiday.cs`) . . . 8
  - 4.1.6 Policy & Compliance (`PolicyDocument.cs` & `PolicyAcknowledgement.cs`) 8

5 RESTful API Specification **8**

6 Deployment & Execution Guide **9**
- 6.1 Fastest Command to Run the Backend . . . 9
- 6.2 Running the Complete Stack Locally . . . 10

6.3 Full-Stack Production Deployment with Docker Compose 10

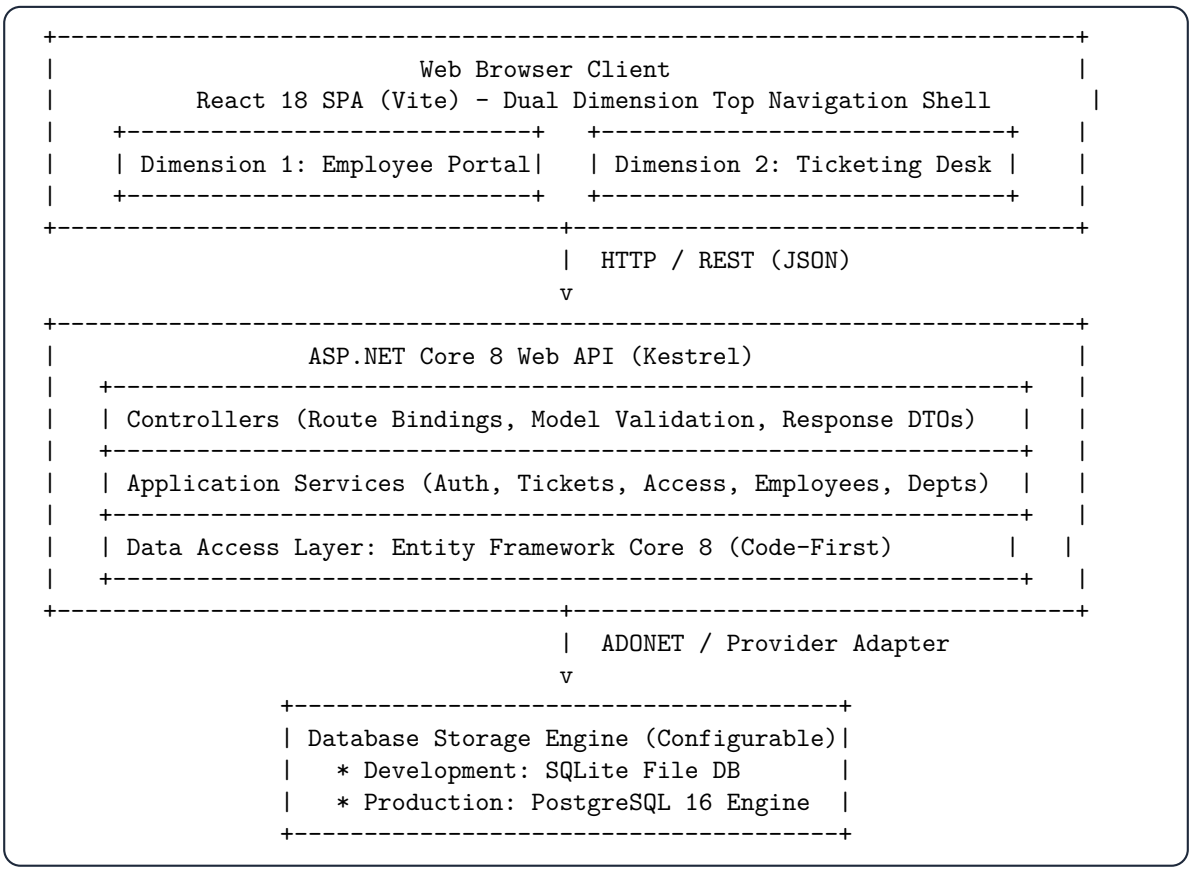
7 Cross-Border Operational Workflows **10**
7.1 Danish-Philippine Operational Alignment 10

8 Documentation Compilation & Maintenance **11**

1 Architectural Overview & Technology Stack

1.1 High-Level System Topology

The platform is designed around clean separation of concerns, decoupling the presentation client from domain logic and backing datastores:



1.2 Technology Stack Breakdown

1.3 Dual Database Provider Strategy

The API implements dynamic provider switching in `Program.cs` based on configuration:

- **SQLite (Default Local)**: Activated when `DatabaseProvider` is omitted or set to `"Sqlite"`. It stores data in `employee_hub.db` and uses `EnsureCreatedAsync()` to guarantee frictionless developer onboarding with zero pre-installed database engines.
- **PostgreSQL (Container / Production)**: Activated when `DatabaseProvider="PostgreSql"`. It connects via Npgsql and automatically applies pending schema migrations via `MigrateAsync()`.

2 Codebase Directory Structure

The monorepo separates concerns between the ASP.NET Core backend service and the React frontend client:

Layer	Technology	Key Characteristics & Implementation
Backend Framework	ASP.NET Core 8 Web API	High-throughput, cross-platform REST service running on C# 12 with native JSON serialization and OpenAPI/Swagger tooling.
ORM / Data Layer	Entity Framework Core 8	Code-First data modelling, automated schema initialization, strongly-typed migrations, and support for dual database providers.
Database Engines	SQLite / PostgreSQL 16	SQLite for instant zero-configuration local runs; PostgreSQL 16 Alpine containerized for transactional multi-user deployment.
Frontend Framework	React 18 (Vite 5)	Modern component architecture, zero heavy external UI dependencies, CSS grid/flexbox layouts, sub-second HMR development.
Networking & State	Native Fetch & Context API	Pure Fetch HTTP client with centralized error interception; React Context for authentication and active RBAC persona switching.
Containerization	Docker & Docker Compose	Multi-stage Docker builds for lean backend binaries and optimized frontend Nginx static file serving with reverse proxy.

Table 1: Technology Stack Specifications

Backend Architecture (ASP.NET Core 8)

```
backend/
|-- Dockerfile
|-- EmployeeHub.slnx
+-- src/EmployeeHub.Api/
    |-- Controllers/
    |   |-- AccessController.cs
    |   |-- AuthController.cs
    |   |-- DepartmentsController.cs
    |   |-- EmployeesController.cs
    |   |-- HardwareController.cs
    |   |-- HolidaysController.cs
    |   |-- PoliciesController.cs
    |   +-- TicketsController.cs
    |-- Services/
    |   |-- Common/ (PasswordHasher)
    |   |-- Interfaces/
    |   +-- Implementations/
    |-- Data/
    |   |-- AppDbContext.cs
    |   +-- DbInitializer.cs
    |-- Models/
    |   |-- Entities/
    |   +-- Enums/
    |-- DTOs/
    |-- Program.cs
    +-- appsettings.json
```

Frontend Architecture (React 18 SPA)

```
frontend/
|-- Dockerfile
|-- nginx.conf
|-- package.json
|-- vite.config.js
+-- src/
    |-- api/
    |   +-- client.js
    |-- context/
    |   +-- AuthContext.jsx
    |-- utils/
    |   +-- workforceStorage.js
    |-- components/
    |   +-- UserProfileNavCard.jsx
    |-- views/
    |   |-- auth/ (AuthView.jsx)
    |   |-- portal/
    |   |   |-- EmployeePortal.jsx
    |   |   |-- ProfileView.jsx
    |   |   |-- AttendanceView.jsx
    |   |   |-- TimeRequestsView.jsx
    |   |   +-- MyTicketsView.jsx
    |   |-- ticketing/
    |   |   |-- TicketingManagement.jsx
    |   |   |-- TicketQueueView.jsx
    |   |   |-- TicketStatsView.jsx
    |   |   +-- DeptManagementView.jsx
    |   |-- CrossBorderCalendarView.jsx
    |   +-- PolicyComplianceView.jsx
    |-- App.jsx
    +-- main.jsx
```

3 Role-Based Access Control (RBAC) Architecture

3.1 Dual-Dimension Conceptual Model

The frontend interface is split into two distinct operational dimensions via the unified header bar:

1. **Dimension 1: Employee Portal (Accessible to ALL)**: Designed for every employee (internal core staff and dedicated external client developers). Features include:
   - **Single-Frame Multi-Box Profile Architecture**: High-density, non-scrollable dashboard organized into two balanced tiers:
     a. *Primary Top Row (3 Box Containers, Equal Height)*:
        - **Left Box**: *Employment Information & Classification* – Name breakdown (Last Name, First Name, Middle Name), Job Role, Client Engagement, Department, Lead, and Hardware Custody indicators.
        - **Middle Box (Wider)**: *Workday Time Clock & Shift Station* – Longer width containing live status, dual cross-border Manila/Copenhagen digital clocks, visual shift progress meter, and 5-action punch buttons (Clock In, Break, Lunch Out, Lunch In, Clock Out).
        - **Right Box**: *Company Announcements & HR Bulletins* – Vertical bulletin cards from HR People Operations.
     b. *Operational Bottom Row (3 Minimal Boxes)*:
        - **Left Minimal Box**: *Onboarding Requirements Fulfillment* – Radial SVG completion meter (75% verified) and statutory clearance status.
        - **Middle Minimal Box (Wider)**: *Pending Requests & Adjustments Summary* – Concise KPI numbers summary (Time Corrections, Overtime Pre-Approvals, Shift Adjustments) and manager routing.
        - **Right Minimal Box**: *Payroll Schedule & Cycle Cut-Offs* – Dynamic single-cycle card displaying Pay Period Date, Filing Cut-Off, Approval Cut-Off, and Crediting Date.
   - **Dedicated Attendance Tab (`AttendanceView.jsx`)**: Full workday biometric history ledger, break/lunch tracking, and monthly timecard metrics.
   - **Dedicated Requests Tab (`TimeRequestsView.jsx`)**: Self-service filing portal for time corrections, overtime pre-approvals, and shift adjustments directed to designated managers.
   - **Calendar & Compliance**: Dual Philippine-Danish holiday calendar and company compliance policy acknowledgements.
2. **Dimension 2: Ticketing System Management (Operational Staff Only)**: Strictly isolated and visible *only* to internal employees who have been granted operational desk access (`HasTicketingAccess = true`). Features a streamlined, 1-word minimalist navigation bar:
   - **Queue**: Triage and manage incoming IT Infrastructure and HR/Legal service requests.
   - **Analytics**: Real-time SLA compliance charts, queue velocity, and priority distribution.
   - **Departments**: Departmental routing configuration and operational unit settings (Admin).
   - **Access**: Operational desk RBAC permission delegation and manager elevation (Staff).
   - **Roster**: Dedicated outstaffing roster and client-developer assignment management.

3.2 Seed Testing Personas Matrix

The system ships pre-seeded with realistic operational personas. All test accounts utilize the default password `Password123!`:

Role / Title	`Work Email`	Dept	Int?	Desk	Privileges & Scope
Super Admin	`superadmin@company.io`	DevOps	Yes	Yes	Full master privileges across all queues, statistical analytics, department CRUD, and user access delegation.
DevOps Admin	`devops@company.io`	DevOps	Yes	Yes	IT & DevOps infrastructure queues; employee access delegation rights.
HR Manager	`hr.manager@company.io`	HR	Yes	Yes	Manages HR, payroll, statutory benefits (SSS/PhilHealth/Pag-IBIG), and onboarding tickets.
IT Specialist	`it.specialist@company.io`	IT Team	Yes	Yes	Hardware provisioning, VPN, asset custody, and technical resolution.
Finance Officer	`finance@company.io`	Finance	Yes	Yes	Equipment purchase reimbursements, statutory deductions, tax documentation.
Dedicated Dev 1	`dev.arnel@company.io`	None	No	No	Dedicated to Danish Client; strictly Employee Portal services only.
Dedicated Dev 2	`dev.bianca@company.io`	None	No	No	Dedicated to Danish Client; strictly Employee Portal services only.

3.3 Granular Authorization Flags

Each employee entity tracks operational capabilities using boolean flags:

- `IsInternal`: Distinguishes internal operations staff from client-dedicated offshore software engineers.
- `HasTicketingAccess`: Controls visibility and entry to Dimension 2 (Ticketing Management).
- `CanAssignEmployees`: Authorizes privileged personnel (Super Admin or DevOps Admin) to grant or revoke ticketing access and delegate operational queues.

4 Domain Entities & Data Models

4.1 Core Entity Definitions

4.1.1 Employee Entity (`Employee.cs`)

Represents both internal core team members and external client-dedicated engineers.

- `Id` (int, PK): Unique identifier.
- `FirstName`, `MiddleName`, `LastName`, `FullName` (string), `WorkEmail` (string, Unique): Identity credentials broken down into discrete name components with combined full name resolution.
- `PasswordHash` (string): PBKDF2/SHA256 salted password hash.
- `JobTitle` (string): Professional role (e.g. Operations Specialist, Account Manager, Developer).
- `PrimaryTechStack` (string, Optional): Professional competencies (omitted from initial sign up credentials to accommodate non-technical staff).
- `EmploymentType` (Enum): `InternalCore (1)` or `ClientDedicated (2)`.
- `Role` (Enum): `AdminHr (1)`, `DanishClientLead (2)`, or `Developer (3)`.
- `TicketingRole` (Enum): `None (0)`, `SuperAdmin (1)`, `DevOpsAdmin (2)`, `DepartmentManager (3)`, or `DepartmentMember (4)`.
- `IsInternal`, `HasTicketingAccess`, `CanAssignEmployees` (bool): Security flags.
- `ClientName` (string?): Client identifier if dedicated (e.g., *"Nordic Logistics ApS"*).
- `DepartmentId` (int?, FK): Associated operational department.

4.1.2 Department Entity (`Department.cs`)

Represents operational units that service internal tickets and manage infrastructure.

- `Id` (int, PK), `Name` (string), `Description` (string), `CreatedAtUtc` (datetime).
- Pre-seeded departments: *Dev Operation*, *IT Team*, *HR/Recruitment*, *Finance*.

4.1.3 Ticket Entity (`Ticket.cs`)

Tracks operational service requests across the enterprise lifecycle.

- `Id` (int, PK), `Title` (string), `Description` (string), `Priority` (string).
- `Queue` (Enum): `ItInfrastructure (1)` or `HrLegal (2)`.
- `Status` (Enum): `Open (1)`, `InProgress (2)`, `Resolved (3)`.
- `CreatedByEmployeeId` (int, FK), `AssignedToEmployeeId` (int?, FK).
- `ResolutionNotes` (string?), `CreatedAtUtc`, `UpdatedAtUtc`.

4.1.4 Hardware Custody Item (`HardwareCustodyItem.cs`)

Tracks company equipment issued to developers.

- `Id` (int, PK), `EmployeeId` (int, FK).
- `ModelName` (string, e.g., *"MacBook Pro 16-inch M3 Max"*).
- `SerialNumber` (string), `AssetTag` (string).
- `AssignedDate` (DateOnly), `ReturnDate` (DateOnly?).
- `Status` (string): `"Active"`, `"InRepair"`, or `"Returned"`.
- `Notes` (string?): Condition reports, peripheral bundles.

4.1.5 Cross-Border Holiday (`Holiday.cs`)

Powers the dual operations calendar to bridge international schedule gaps.

- `Id` (int, PK), `Title` (string), `HolidayDate` (DateOnly).
- `Country` (Enum): `Philippines (1)` or `Denmark (2)`.
- `IsWorkingDay` (bool): Accommodates special working holidays and local observances.

4.1.6 Policy & Compliance (`PolicyDocument.cs` & `PolicyAcknowledgement.cs`)

Maintains audit logs of legal and regulatory compliance.

- `PolicyDocument`: `Title`, `Version`, `SummaryMarkdown`, `IsMandatory`, `PublishedAtUtc`.
- `PolicyAcknowledgement`: Tracks employee ID, policy ID, and digital signature timestamp.

5 RESTful API Specification

All endpoints are hosted under `/api/*`. The service includes interactive OpenAPI documentation at `/swagger`.

`HTTP Method & Path`	Controller Layer	Description & Parameters
Authentication & Personas (`AuthController`)		
`POST /api/auth/login`	AuthController	Authenticates user by email/password; returns user identity and roles.
`POST /api/auth/register`	AuthController	Registers standard employee with default developer privileges.
`GET /api/auth/personas`	AuthController	Returns pre-configured testing personas for 1-click dev switching.
Employee Directory (`EmployeesController`)		
`GET /api/employees`	EmployeesController	Lists all employees; supports search and department filters.
`GET /api/employees/{id}`	EmployeesController	Retrieves comprehensive profile including hardware and tickets.
`POST /api/employees`	EmployeesController	Creates a new employee record.
`PUT /api/employees/{id}`	EmployeesController	Updates employee details, tech stack, and client association.
`GET /api/employees/{id}/hardware`	EmployeesController	Lists physical assets assigned to the specified employee.
Service Desk & Ticketing (`TicketsController`)		
`GET /api/tickets`	TicketsController	Lists tickets; filterable by `status`, `queue`, and `employeeId`.
`GET /api/tickets/stats`	TicketsController	Computes operational metrics, queue counts, and resolution stats.
`GET /api/tickets/{id}`	TicketsController	Retrieves complete ticket details with history and comments.

HTTP Method & Path	Controller Layer	Description & Parameters
`POST /api/tickets`	TicketsController	Submits a new support or operational request.
`PATCH /api/tickets/{id}/status`	TicketsController	Updates ticket status (`Open` → `InProgress` → `Resolved`).
`PATCH /api/tickets/{id}/assign`	TicketsController	Assigns ticket to an operational team member.
Access Delegation (`AccessController`)		
`GET /api/access/employees`	AccessController	Returns employee roster with ticketing roles and security flags.
`PATCH /api/access/{id}`	AccessController	Updates `HasTicketingAccess`, `TicketingRole`, and delegation rights.
Hardware Custody (`HardwareController`)		
`GET /api/hardware`	HardwareController	Lists all tracked corporate hardware assets across the organization.
`POST /api/hardware`	HardwareController	Registers new hardware asset custody assignment.
Holidays & Cross-Border Operations (`HolidaysController`)		
`GET /api/holidays`	HolidaysController	Returns combined Philippine and Danish holiday schedule.
`POST /api/holidays`	HolidaysController	Adds custom company calendar event or working day exception.
Policies & Compliance (`PoliciesController`)		
`GET /api/policies`	PoliciesController	Lists active corporate policies and acknowledgement mandates.
`POST /api/policies/{id}/acknowledge`	PoliciesController	Logs employee timestamped policy compliance acknowledgement.

6 Deployment & Execution Guide

6.1 Fastest Command to Run the Backend

The backend API can be launched with a single command from the project root:

```
# Option A: Via npm script defined in package.json
npm run start:backend

# Option B: Direct .NET 8 CLI execution
dotnet run --project backend/src/EmployeeHub.Api --launch-profile http
```

Zero-Config SQLite Initialization

When launched using the `http` profile, the API binds to `http://localhost:5000` and immediately provisions the local SQLite database (`employee_hub.db`) complete with pre-seeded personas, departments, policies, and tickets. No database installation is required.

6.2 Running the Complete Stack Locally

To run both backend and frontend concurrently for local development:

1. **Start the Backend API**:

```
npm run start:backend
```

   API is live at http://localhost:5000; Swagger UI at http://localhost:5000/swagger.

2. **Start the React Frontend**: In a separate terminal window:

```
npm --prefix frontend run dev
```

   The Vite dev server will launch at http://localhost:5173.

6.3 Full-Stack Production Deployment with Docker Compose

For staging or production deployments running PostgreSQL 16:

```
docker compose up --build
```

Service	External Address	Internal Architecture
Frontend Web	http://localhost:8080	Nginx Alpine serving optimized React production bundle with reverse-proxy.
Backend API	http://localhost:5000	ASP.NET Core 8 Alpine container running in `Development` mode.
Database Engine	`localhost:5432`	PostgreSQL 16 Alpine container with persistent named volume `pgdata`.

Table 4: Docker Compose Service Endpoints

7 Cross-Border Operational Workflows

7.1 Danish-Philippine Operational Alignment

The system specifically addresses the logistical challenges of offshore software teams dedicated to Danish client operations:

- **Dual Operations Calendar**: Displays simultaneous Philippine and Danish public holidays side-by-side. Highlights *Special Working Holidays* and non-overlapping days where coverage planning is critical (e.g., Danish *Store Bededag* or Philippine *Araw ng Kagitingan*).
- **Hardware Custody Receipts**: Maintains transparent custody records of premium corporate hardware issued to offshore developers (asset serial tags, MacBook model numbers, delivery dates, and return condition logs).
- **Statutory Remittance & HR Requests**: Dedicated routing for Philippine legal compliance tickets (Certificate of Employment requests, SSS, PhilHealth, Pag-IBIG inquiries, and tax form 2316 processing).
- **Client Dedicated Isolation**: Engineers dedicated to Danish clients are restricted from viewing other client teams or internal departmental ticketing queues, ensuring strict client confidentiality.

8 Documentation Compilation & Maintenance

This documentation is maintained in LaTeX format (`documentation/main.tex`). To recompile into PDF format:

```
# Recompile using pdflatex from the documentation folder:
cd documentation
pdflatex -interaction=nonstopmode main.tex
pdflatex -interaction=nonstopmode main.tex    # Run twice to resolve TOC
    references

# Or using latexmk (automated multi-pass):
latexmk -pdf main.tex
```

Document Summary

Document Name: `main.tex` → `main.pdf`
Output Directory: `proposal-employee-hub/documentation/`
Maintainer: Enterprise Operations Hub Core Team